浙江大学

本科实验报告

课程名称：	实验 11 异步输入的同步器和开关防颤动电路的设计
姓　　名：	
学　　院：	信息与电子工程学院
系：	
专　　业：	信息工程
学　　号：	
指导教师：	屈民军，唐奕

2025 年 5 月 7 日

专业：信息工程
姓名：
学号：
日期：2025.5.7
地点：东四

浙江大学实验报告

课程名称：数字系统设计实验
指导老师：屈民军，唐奕
成绩：
实验名称：实验 11 异步输入的同步器和开关防颤动电路的设计

一、实验目的

1．掌握减少亚稳态的方法，了解亚稳态给系统带来的危害

2．掌握开关防颤动的概念和消除的方法

3．初步了解控制器的设计

二、概述与实验任务

1．概述：异步信号主要有两种，第一是开关的输入，第二是来自不同时钟域的由不同时钟同步的信号

（1）异步输入不是总能满足建立和保持时间的要求，异步输入常常会把错误的数据锁存到触发器，或进入亚稳态，该状态下触发器输出不能识别为 1 或 0

（2）异步输入信号的宽度是不确定的

（3）开关输入，当弹簧开关被按下时，机械触点会振动产生不稳定信号

2．实验任务

（1）设计一个按键处理模块，要求每按一次按键，按键处理模块输出一个正脉冲信号，脉冲的宽度为一个 clk 时钟周期

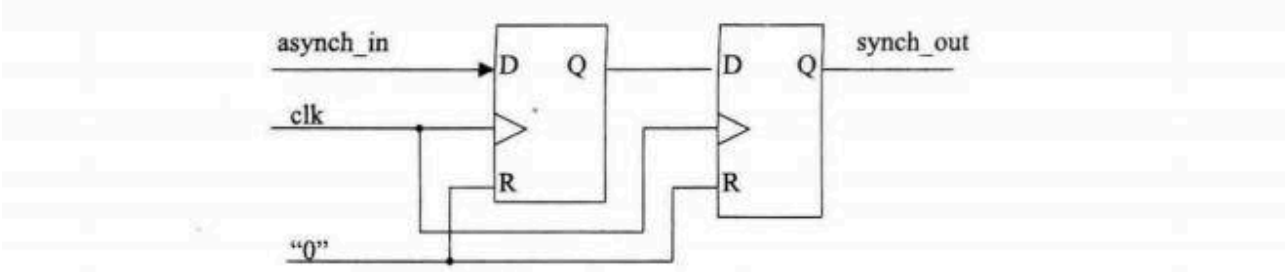
（2）搭建一个按键处理模块测试电路，每按键一次 led 指示灯改变状态

三、实验设计原理与 Verilog 代码

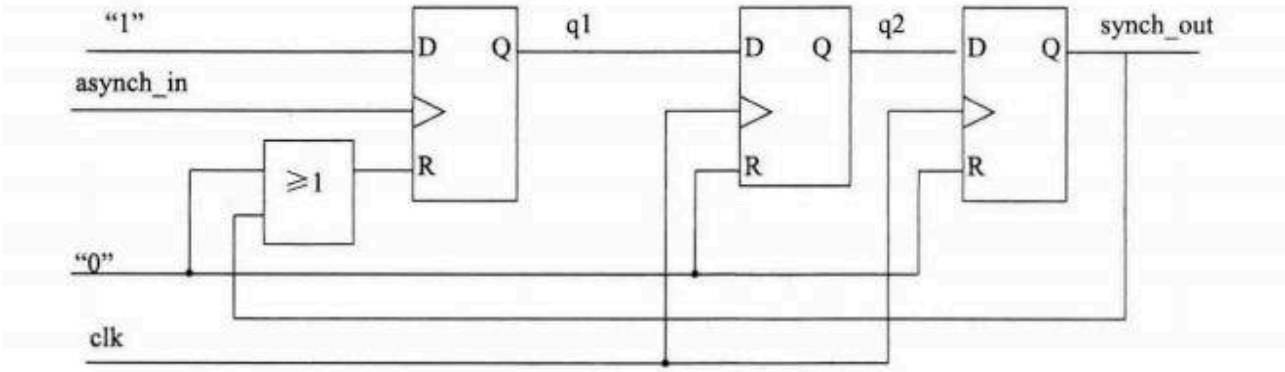
1．同步器设计

（1）原理：避免亚稳态设计思路：首先尽可能减少出现亚稳态的可能性，其

次尽可能减少亚稳态给系统带来危害的可能性，故采用双锁存器法，即将输入异步信号用两个锁存器连续锁存两次



(a) 当异步脉冲宽度大于时钟周期时的电路



(b) 当异步脉冲宽度小于时钟周期时的电路

（2）代码：即调用两次 D 触发器

```
module synch(asynch_in,clk,synch_out);
    input clk,asynch_in;
    output synch_out;
    wire temp;
    dffre #(.n(1))ff1(.d(asynch_in),.en(1),.r(1'b0),.clk(clk),.q(temp));
    dffre #(.n(1))ff2(.d(temp),.en(1'b1),.r(1'b0),.clk(clk),.q(synch_out));
endmodule
```

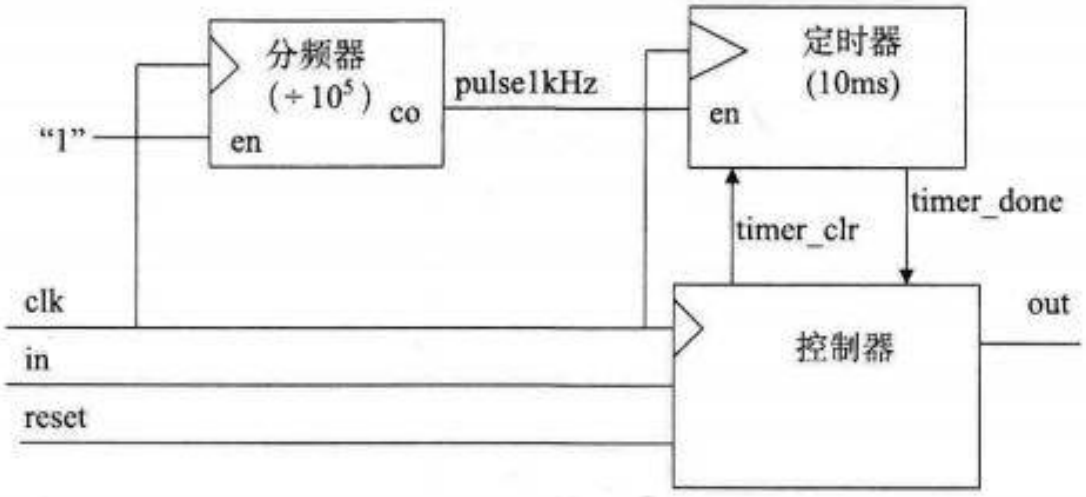
2．开关防颤动电路

（1）开关的颤动：人们完成一次按键操作的指压力，从最初按下到接触稳定要经过数毫秒的颤动，按键松开也会出现同样的问题，出现的不稳定问题表现在电平在 0、1 之间无规则摆动

（2）设计原理：关键是避免在颤动期采样，颤动期时间长短与开关类型和指压力有关。控制期是防颤动电路的核心，一般情况电路在 LOW 状态下等待，

当按键按下(检测到 in=1)，电路转到 WAIT_HIGH 状态，启动 10ms 定时器；当 10ms 定时结束(timer_done=1)，电路进入 HIGH 状态，采样开关输入，等待按键释放，当按键释放(检测到 in=0)，电路转到 WAITLOW 状态，启动 10ms 定时器，消除释放颤动期；定时结束后回到 LOW 状态下等待下次按键操作。

（3）电路图：主要由分频器得到 1kHz，10ms 定时器和控制器组成



（4）代码：尤其注意控制器的实现，需要有异步置零端

```
module debouncer(in,clk,reset,v0);
    parameter sim=0;
    input clk,in,reset;
    output v0;

    wire pulse1kHz,timer_done;
    reg timer_clr,v0;
    reg[1:0] state,next_state;
    counter_n #(.n(sim?32:10**5),.counter_bits(17))DivI(.clk(clk),.r(1'b0),.en(1'b1),.co(pulse1kHz),.q());//分频器
    timer #(.n(10),.counter_bits(4))TimerInst(.clk(clk),.r(timer_clr),.en(pulse1kHz),.done(timer_done));//10ms计时器

    //对状态进行编码
    parameter LOW=0,WAIT_HIGH=1,HIGH=2,WAIT_LOW=3;
    //判断异步置0端
    always @(posedge clk)
        begin
            if(reset) state=LOW;
            else state=next_state;
        end

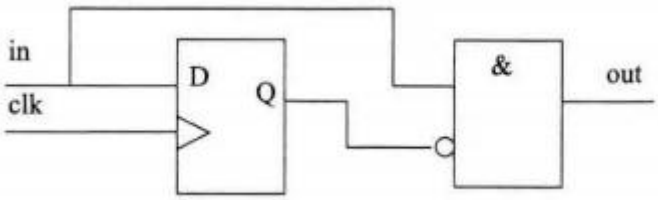
    //控制器部分,根据in, 现态, timer_done决定次态
    always@(*)
        begin
            timer_clr=0;
            v0=0;
        case(state)
```

```
            LOW:
                begin
                    timer_clr=1;
                    v0=0;
                    next_state=(in==1)?WAIT_HIGH:LOW;
                end
            WAIT_HIGH:
                begin
                    timer_clr=0;
                    v0=1;
                    next_state=(timer_done==1)?HIGH:WAIT_HIGH;
                end
            HIGH:
                begin
                    timer_clr=1;
                    v0=1;
                    next_state=(in==1)?HIGH:WAIT_LOW;
                end
            WAIT_LOW:
                begin
                    timer_clr=0;
                    v0=1;
                    next_state=(timer_done==1)?LOW:WAIT_LOW;
                end
        endcase
        end
endmodule
```

3．脉冲变宽电路

（1）原理：从上面分析得知，开关或按键的输入信号宽度远远大于一个时钟周期，脉宽变换电路的作用是将输入信号宽度变换成一个时钟周期。由于输入已是同步的且宽度大于一个时钟的脉冲，因此脉宽变换电路的设计就变得非常简单。

（2）电路图



（3）代码

```
module pulse_variation(in,clk,out);
    input in,clk;
    output out;
    wire temp;
    dffre #(.n(1))ff(.d(in),.en(1'b1),.r(1'b0),.clk(clk),.q(temp));
    assign out=in&(~temp);
endmodule
```

4．顶层代码

（1）开关防颤动代码：分别调用同步器，防颤动电路，脉冲变换电路三个模块，由于存在大分频比的分频器，在顶层代码中设置参数 sim，sim=0 时综合实现，实现所需的分频比，sim=1 时进行仿真，调小分频比，加快仿真速度

```
module button_process_unit(reset,ButtonIn,clk,ButtonOut);
    input clk,reset,ButtonIn;
    output ButtonOut;
    parameter sim=100000;

    //调用同步器模块
    wire temp1;
    synch synch_inst(.asynch_in(ButtonIn),.clk(clk),.synch_out(temp1));
    //调用防颤电路
    wire temp2;
    debouncer #(.sim(sim)) debouncer_inst(.in(temp1),.clk(clk),.reset(reset),.v0(temp2));
    //调用脉冲变宽
    pulse_variation pulse_variation_inst(.in(temp2),.clk(clk),.out(ButtonOut));
endmodule
```

（2）上传开发板代码（含 led）

```
module button_process_unit_led(reset,ButtonIn,clk,led);
    input clk,reset,ButtonIn;
    output led;
    parameter sim=100000;

    //调用同步器模xE5x9D?
    wire temp1;
    synch synch_inst(.asynch_in(ButtonIn),.clk(clk),.synch_out(temp1));
    //调用防颤电路
    wire temp2;
    debouncer #(.sim(sim)) debouncer_inst(.in(temp1),.clk(clk),.reset(reset),.v0(temp2));
    //调用脉冲变宽
    wire ButtonOut;
    pulse_variation pulse_variation_inst(.in(temp2),.clk(clk),.out(ButtonOut));
    //外界测试电路
    dffre #(.n(1))ff(.d(~led),.en(ButtonOut),.r(1'b0),.clk(clk),.q(led));
endmodule
```

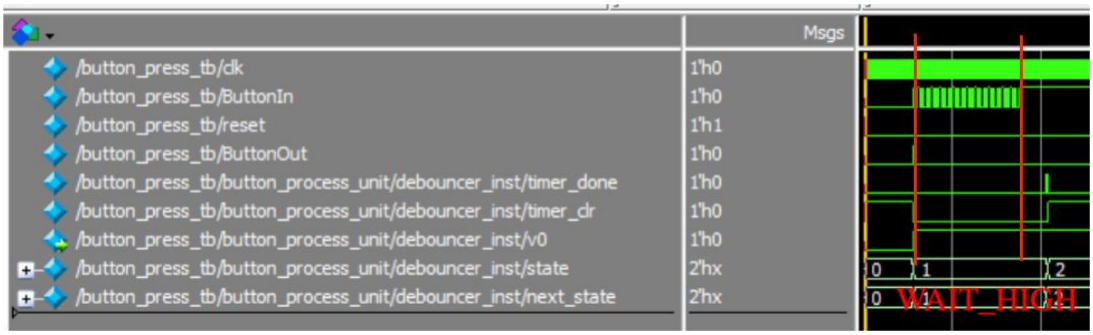
四、ModelSim 的仿真

1．执行 button_process_tb 后得到的 wave

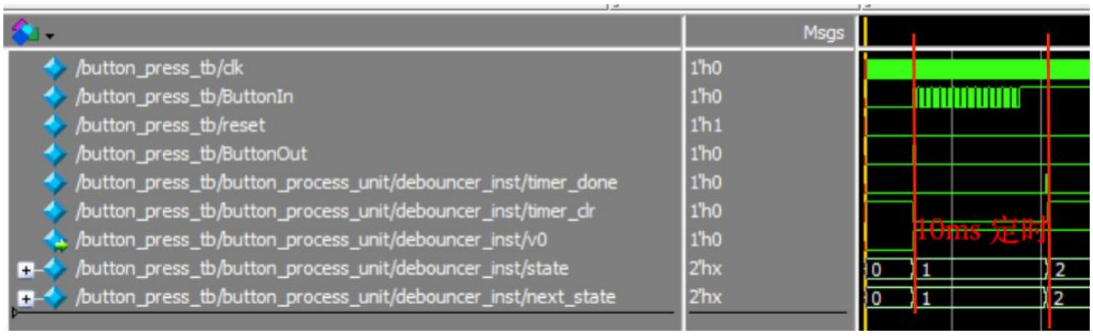


2．具体分析：

（1）开关颤动，检测到 in=1，state 进入 1（WATI_HIGH）等待采样，并且 timer_clr=1，使能 10ms 定时器，开始计时，v0=1

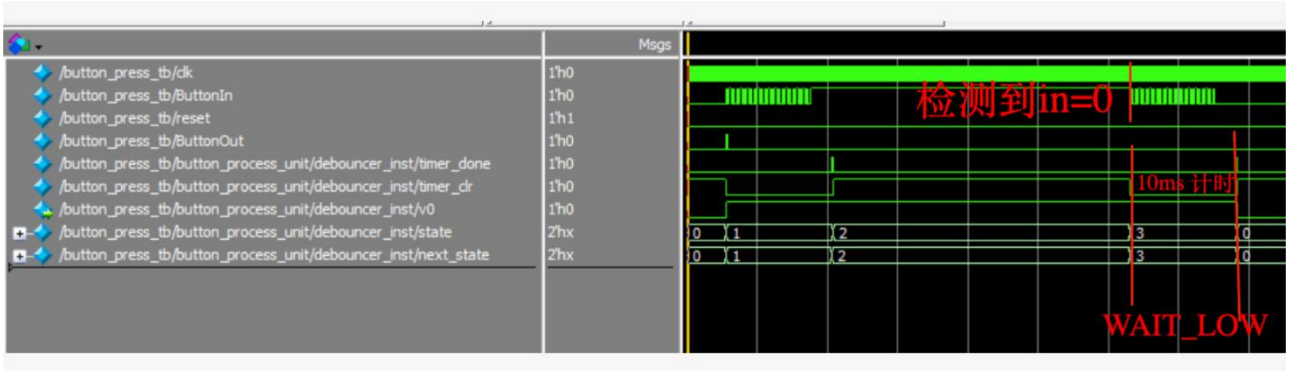


（2）10ms 定时完成后，state 进入 2（HIGN），开始采样开关输入，等待按键释放，此时 timer_clr=0，10ms 计时器不工作，v0=1；可以看出此时 ButtonIn 为 1，因此采样开关输入为 1，v0=1



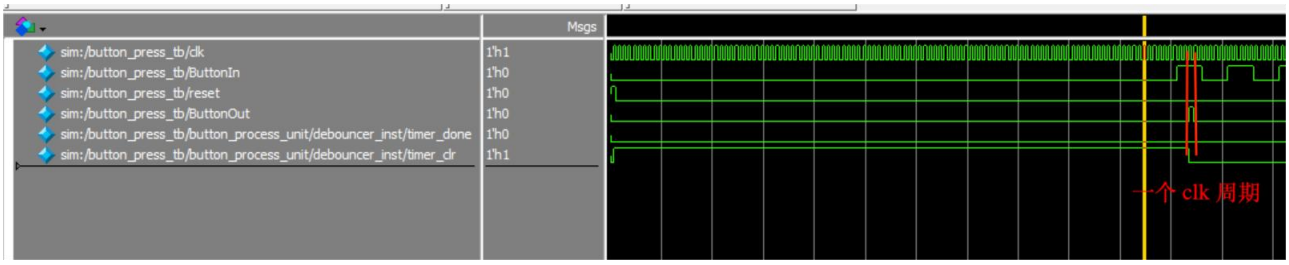


（3）开关颤动，检测到 in=0，state 进入 3（WAIT_LOW），timer_clr=1，使能 10ms 计时器，开始计时，消除按键释放颤动期



（4）10ms 定时结束后进入 LOW 状态，等待下一次按键操作

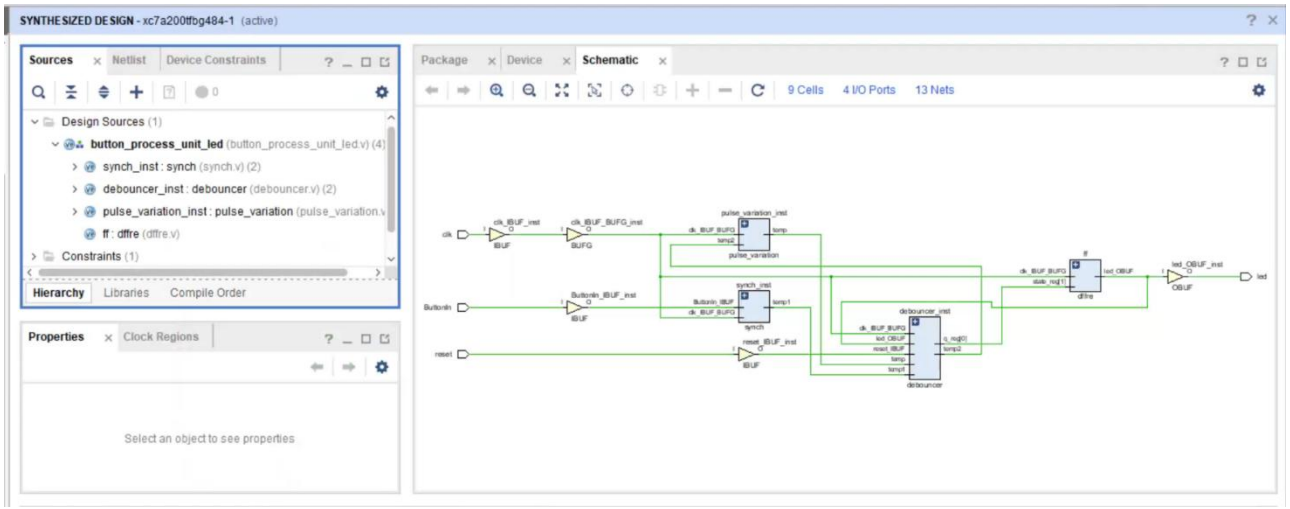
（5）脉冲变换：可以看出开关或按键的输入信号宽度远远大于一个时钟周期，脉冲变换电路作用是将输入信号变换成一个时钟周期

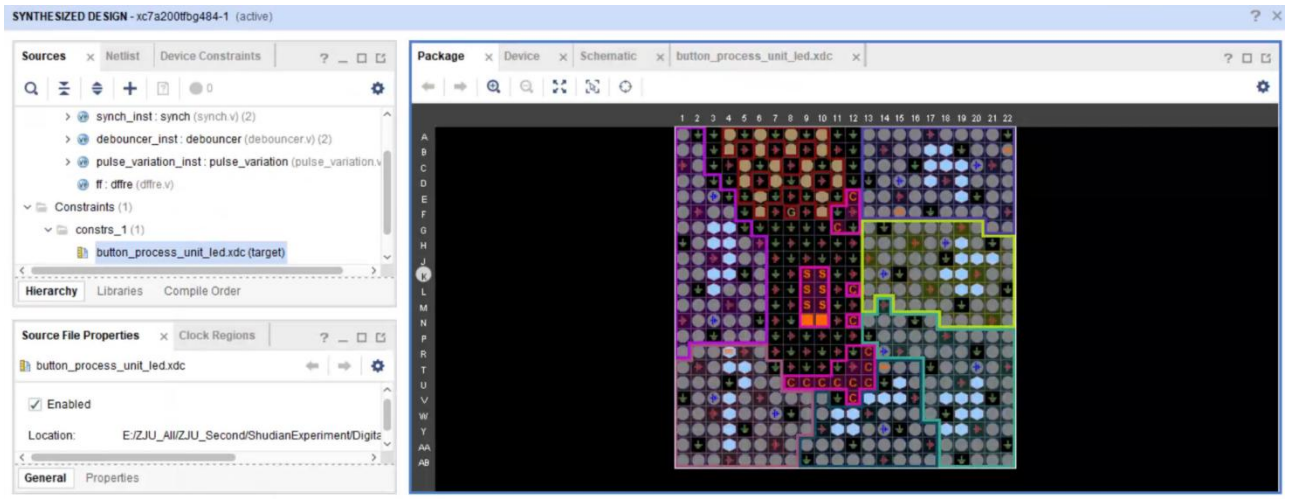


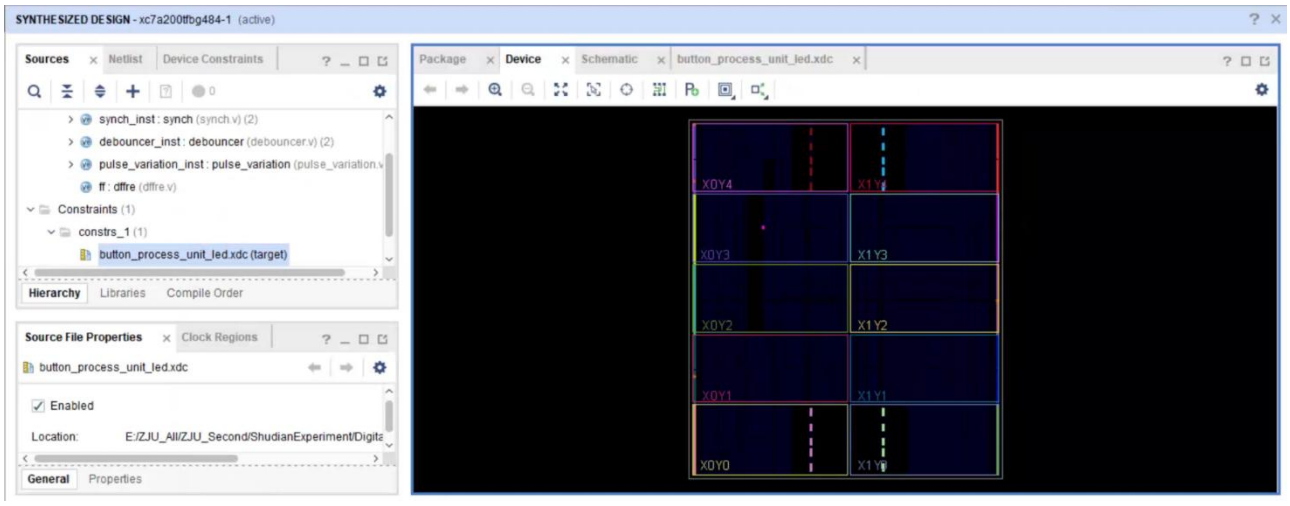
五、实验结果和开发板测试

1．vivado 工程文件建立

（1）RTL 分析与综合







（2）引脚约束：按照课本要求进行约束，特别注意 led 是 T14

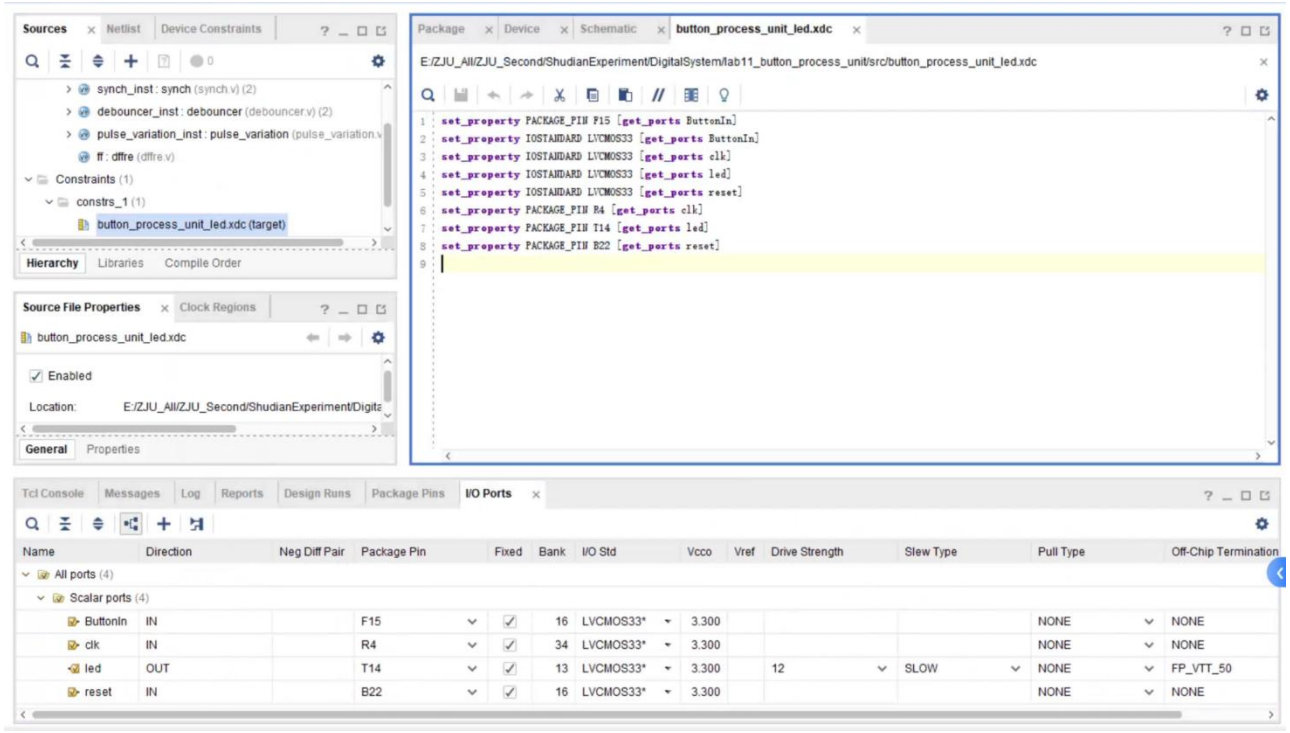
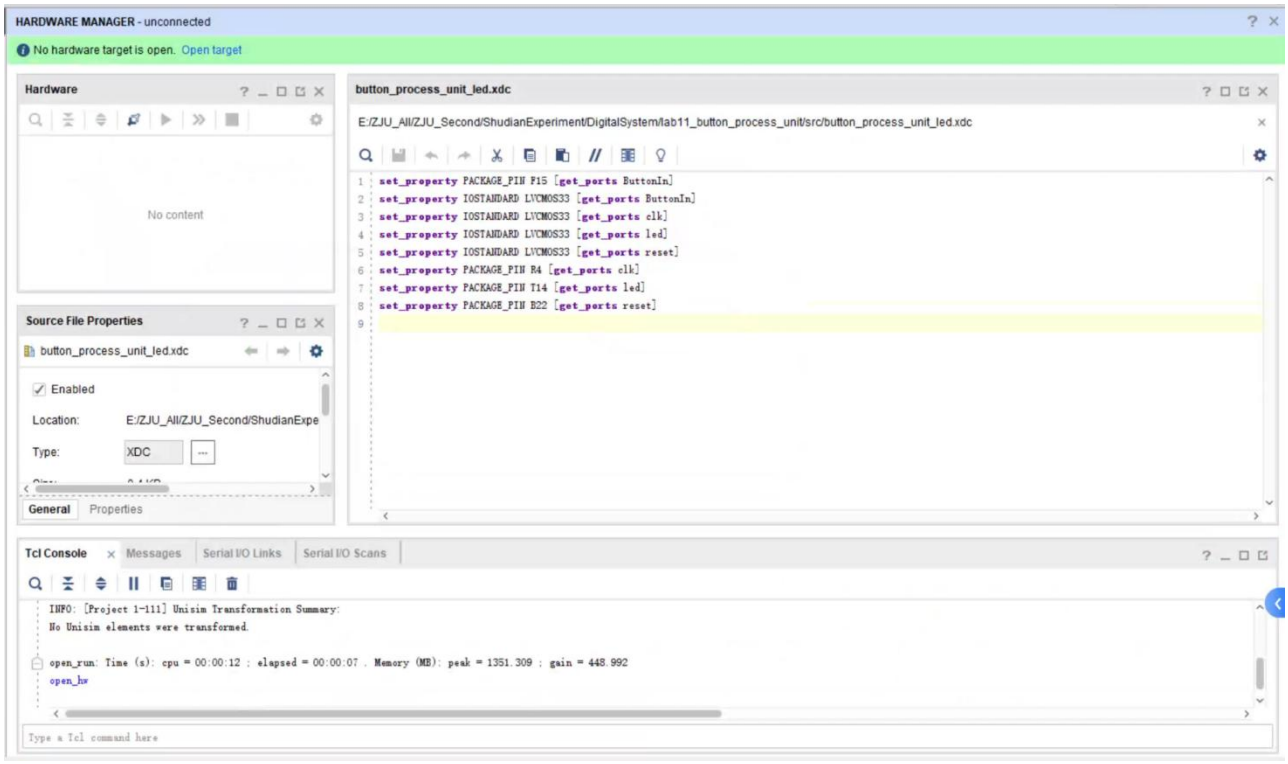


表 5.16　FPGA 引脚约束内容

引脚名称	I/O	引脚编号 (Nexys Video)	引脚编号 (Basys 3)	电气特性	说明
clk	Input	R4	W5	LVCMOS33	系统 100MHz 主时钟
reset	Input	B22	U18	LVCMOS33	开发板上的中间按键
ButtonIn	Input	F15	T18	LVCMOS33	开发板上的上边按键
led	Output	T14	U16	LVCMOS33	LED0 指示灯

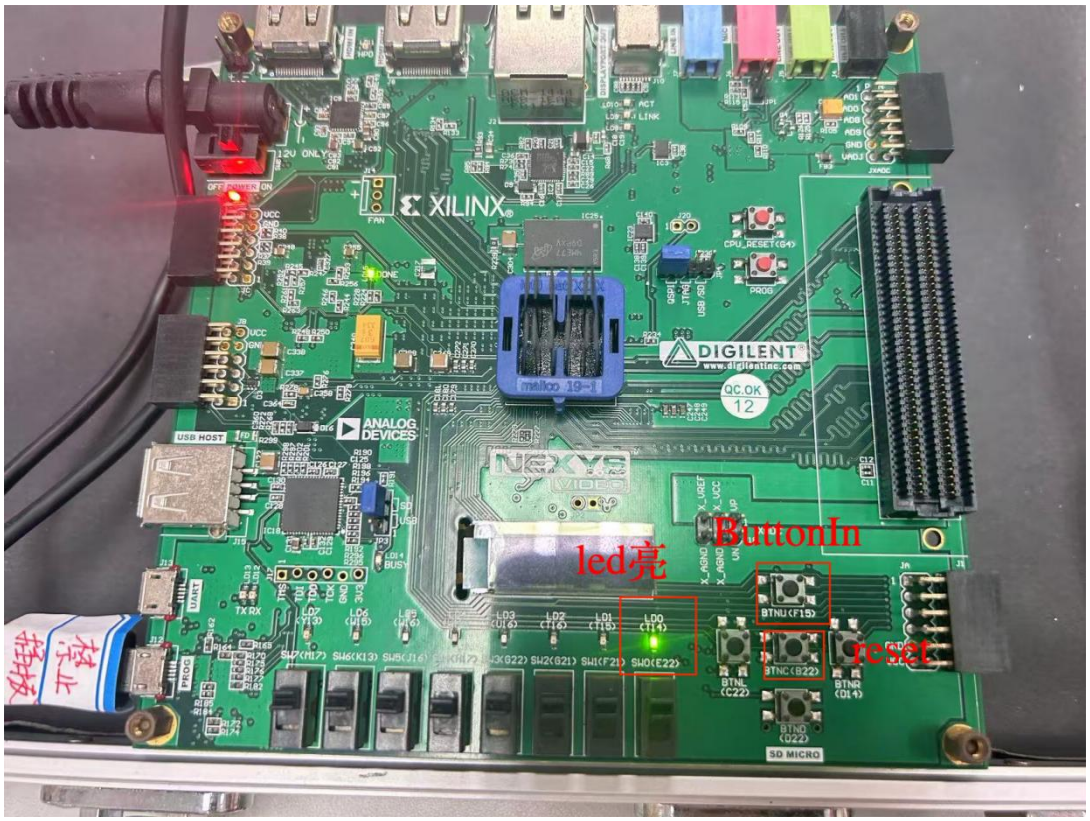
（3）连接开发板



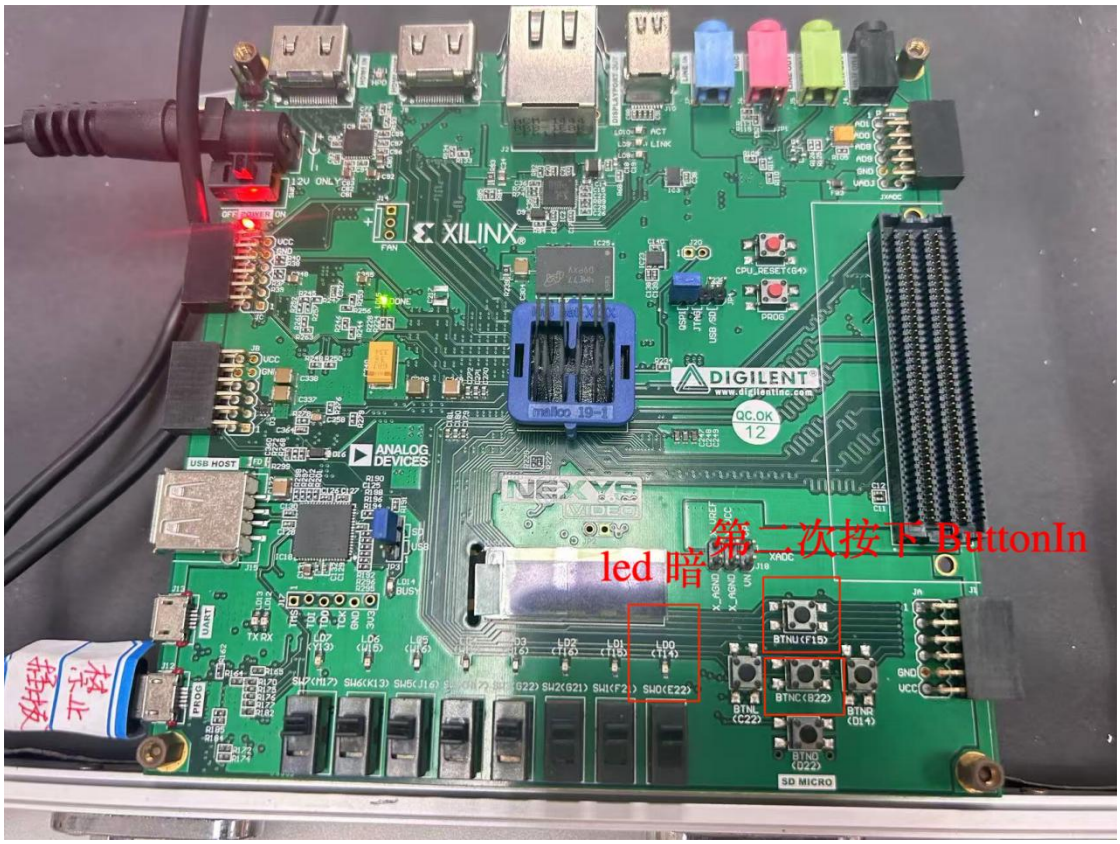
2．开发板测试：以下为 button_process_unit 防颤动功能测试

（1）引脚分析，reset 按键为开发板中间按键，用于复位；ButtonIn 为开发板上上边按键，用于按键信号输入；led 为输出，为 LED0 指示灯，在测试过程中表示 ButtonOut 的输出情况

（2）测试 1：第一次按下 ButtonIn 按键，观察 LED0 的情况，可以看出：尽管按键开关时有所颤动，但是电路仍然可以精准识别开关按下 in=1，LED0 亮



（3）测试 2：第二次按下 ButtonIn 按键，观察 LED0 的情况，可以看出：尽管按键开关时有所颤动，但是电路仍然可以精准识别开关按下 in=1，LED0 灭



六、实验过程中遇到的问题和解决方案

1．第一个问题是遇到 ButtonOut 没有输出，经过逐个模块检查的时候，发现是防颤动电路这一模块没有输出，通过加入其余变量才发现，又发生了和上一次一样的问题，就是在这个模块中又单独声明了一个变量为 out 且没有与 v0 相连，没有观察到电路图 v0 其实就是这一模块的输出，导致 out 没有输出

2．第二个问题是我的同桌询问我的问题，一开始我也觉得他的代码没有任何的问题，但是在仿真过程中，ButtonOut 一直没有信号，逐个模块检查的时候发现也是控制器这边出现了问题，检查过程中发现 state 和 next_state 都是不定态，经过询问老师后才解决，老师适当在 tb 文件中 delay*3，我们才知道这是因为信号传递需要时间导致前级信号没传来，出现不定态

（1）首先是没有对触发器进行初始化，由于控制器前级是同步器电路，主要用到了 D 触发器，可以将 D 触发器的 q 初始化为 0，这样就不会出现不定态的传播

（2）第二个原因是代码的问题，本代码中使用了（in==1）？条件赋值，这个语句如果 in 确实为 1，则将前面的值赋出，但是如果不是 1，包括 in=0 和 in 为不定态，部分软件版本会将两者出现时都将后面的值赋出，但是有的软件只会处理 in=0，而 in 为不定态则处理为不定态，导致 next_state 为不定态

```
begin
    timer_clr=1;
    v0=0;
    next_state=(in==1)?WAIT_HIGH:LOW;
end
```

（3）这个问题出现有很多原因，可能与软件的版本有关导致，也有其他各种原因，较为复杂，这个问题的出现确实需要很多经验才可以发现，主要还是考察时序电路的传输延迟时间、建立时间和保持时间之间的复杂关系

3．第三个问题也是我在帮助同桌修改时发现的，上传板子验证时，发现无论按哪个按键都无法进行使得 LED0 亮，后续在老师帮助检查下，发现代码和电路基本框架是没有问题的，但是在 IO 端口配置的时候，reset 配到了错误的按键，而由于这个未知按键的高低电平未知，结合 LED0 一直处于熄灭状态，因此这个 reset 大概率一直处于高电平状态，使得整个电路一直处于复位状态。

七、思考题

1．同步器的锁存器级数与什么因素有关？

答：亚稳态的概率容忍度：级数越多，亚稳态传播到后续逻辑的概率越低。系统时钟频率：高频系统需要更多级数来降低亚稳态风险。输入信号变化率：信号变化越频繁，亚稳态概率越高，需更多级数。可靠性要求：高可靠性系统（如航空航天）可能采用三级或更多锁存器。

2．在本实验中，如果对 UP 开关信号不作脉冲变换处理，那将造成什么结果？

答：多次误触发：由于 UP 信号较宽，单次按键持续多个 clk，使得后续电路多次响应同一按键操作。逻辑错误：例如 LED 可能在单次按键中多次翻转，破坏预期功能。

3．为什么对系统复位信号 reset 不需要进行防颤动处理？

答：电平敏感：复位信号只需在有效电平保持足够时间即可完成复位，无需捕捉边沿。因此不需要进行防颤动处理